

NoteBook

December 4, 2025

PROJET 4 DATA ANALYST

Réalisez une étude de santé publique avec R ou Python

1 OBJECTIF DE CE NOTEBOOK

Bienvenue dans l'outil plébiscité par les analystes de données Jupyter.

Il s'agit d'un outil permettant de mixer et d'alterner codes, textes et graphique.

Cet outil est formidable pour plusieurs raisons:

- il permet de tester des lignes de codes au fur et à mesure de votre rédaction, de constater immédiatement le résultat d'une instruction, de la corriger si nécessaire.
- De rédiger du texte pour expliquer l'approche suivie ou les résultats d'une analyse et de le mettre en forme grâce à du code html ou plus simple avec **Markdown**
- d'agrémenter de graphiques

Pour vous aider dans vos premiers pas à l'usage de Jupyter et de Python, nous avons rédigé ce notebook en vous indiquant les instructions à suivre.

Il vous suffit pour cela de saisir le code Python répondant à l'instruction donnée.

Vous verrez de temps à autre le code Python répondant à une instruction donnée mais cela est fait pour vous aider à comprendre la nature du travail qui vous est demandée.

Et garder à l'esprit, qu'il n'y a pas de solution unique pour résoudre un problème et qu'il y a autant de résolutions de problèmes que de développeurs ;)...

Etape 1 - Importation des librairies et chargement des fichiers

1.1 - Importation des librairies

```
[57]: #Importation de la librairie Pandas  
import pandas as pd
```

1.2 - Chargement des fichiers Excel

```
[58]: #Importation des fichiers csv  
  
population = pd.read_csv('population.csv')  
dispo_alimentaire = pd.read_csv('dispo_alimentaire.csv')  
aide_alimentaire = pd.read_csv('aide_alimentaire.csv')
```

```
sous_nutrition = pd.read_csv('sous_nutrition.csv')
```

Etape 2 - Analyse exploratoire des fichiers

2.1 - Analyse exploratoire du fichier population

```
[59]: #Afficher les dimensions du dataset
print("Le tableau comporte {} observations ou articles".format(population.
    ↪shape[0]))
print("Le tableau comporte {} colonnes".format(population.shape[1]))
```

Le tableau comporte 1416 observations ou articles

Le tableau comporte 3 colonnes

```
[60]: #Consulter le nombre de colonnes
print(population.shape[1])

#La nature des données dans chacune des colonnes
print(population.dtypes)

#Le nombre de valeurs présentes dans chacune des colonnes
display(population.count())
```

```
3
Zone      object
Année     int64
Valeur    float64
dtype: object

Zone      1416
Année     1416
Valeur    1416
dtype: int64
```

```
[61]: #Affichage les 5 premières lignes de la table
display(population.head())
```

	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603
3	Afghanistan	2016	35383.032
4	Afghanistan	2017	36296.113

```
[62]: #changement du nom de la colonne Valeur par Population
population = population.rename(columns={'Valeur': 'Population'})
print(population.columns.tolist())
```

```
['Zone', 'Année', 'Population']
```

```
[63]: #Nous allons harmoniser les unités. Pour cela, nous avons décidé de multiplier
      ↪ la population par 1000
```

```
population['Population'] *= 1000
```

```
[64]: #Affichage les 5 premières lignes de la table pour voir les modifications
display(population.head())
```

	Zone	Année	Population
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

2.2 - Analyse exploratoire du fichier disponibilité alimentaire

```
[65]: #Afficher les dimensions du dataset
print(f"Le tableau comporte {dispo_alimentaire.shape[0]} observations ou
      ↪ articles")
print(f"Le tableau comporte {dispo_alimentaire.shape[1]} colonnes")
```

Le tableau comporte 15605 observations ou articles

Le tableau comporte 18 colonnes

```
[66]: #Affichage les 5 premières lignes de la table
display(dispo_alimentaire.head())
```

	Zone	Produit	Origine	Aliments pour animaux	\
0	Afghanistan	Abats Comestible	animale		NaN
1	Afghanistan	Agrumes, Autres	vegetale		NaN
2	Afghanistan	Aliments pour enfants	vegetale		NaN
3	Afghanistan	Ananas	vegetale		NaN
4	Afghanistan	Bananes	vegetale		NaN

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
0	NaN		5.0
1	NaN		1.0
2	NaN		1.0
3	NaN		0.0
4	NaN		4.0

	Disponibilité alimentaire en quantité (kg/personne/an)	\
0	1.72	
1	1.29	
2	0.06	
3	0.00	
4	2.70	

	Disponibilité de matière grasse en quantité (g/personne/jour) \
0	0.20
1	0.01
2	0.01
3	NaN
4	0.02

	Disponibilité de protéines en quantité (g/personne/jour) \
0	0.77
1	0.02
2	0.03
3	NaN
4	0.05

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité \
0	53.0	NaN	NaN
1	41.0	2.0	40.0
2	2.0	NaN	2.0
3	0.0	NaN	0.0
4	82.0	NaN	82.0

	Nourriture	Pertes	Production	Semences	Traitement	Variation de stock
0	53.0	NaN	53.0	NaN	NaN	NaN
1	39.0	2.0	3.0	NaN	NaN	NaN
2	2.0	NaN	NaN	NaN	NaN	NaN
3	0.0	NaN	NaN	NaN	NaN	NaN
4	82.0	NaN	NaN	NaN	NaN	NaN

```
[67]: #remplacement des NaN dans le dataset par des 0
dispo_alimentaire = dispo_alimentaire.fillna(0)
```

```
[68]: #multiplication de toutes les lignes contenant des milliers de tonnes en Kg
```

```
FACTEUR_KT_VERS_KG = 1000000

colonnes_a_convertir = [
    'Aliments pour animaux',
    'Autres Utilisations',
    'Disponibilité intérieure',
    'Exportations - Quantité',
    'Importations - Quantité',
    'Nourriture',
    'Pertes',
    'Production',
    'Semences',
    'Traitement',
```

```

'Variation de stock']

for colonne in colonnes_a_convertir:
    dispo_alimentaire.loc[:, f'{colonne}_kg'] = dispo_alimentaire[colonne] *
↳FACTEUR_KT_VERS_KG

```

```

[69]: #Affichage les 5 premières lignes de la table
display(dispo_alimentaire.head())

```

	Zone	Produit	Origine	Aliments pour animaux \
0	Afghanistan	Abats Comestible	animale	0.0
1	Afghanistan	Agrumes, Autres	vegetale	0.0
2	Afghanistan	Aliments pour enfants	vegetale	0.0
3	Afghanistan	Ananas	vegetale	0.0
4	Afghanistan	Bananes	vegetale	0.0

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour) \
0	0.0	5.0
1	0.0	1.0
2	0.0	1.0
3	0.0	0.0
4	0.0	4.0

	Disponibilité alimentaire en quantité (kg/personne/an) \
0	1.72
1	1.29
2	0.06
3	0.00
4	2.70

	Disponibilité de matière grasse en quantité (g/personne/jour) \
0	0.20
1	0.01
2	0.01
3	0.00
4	0.02

	Disponibilité de protéines en quantité (g/personne/jour) \
0	0.77
1	0.02
2	0.03
3	0.00
4	0.05

	Disponibilité intérieure ...	Autres Utilisations_kg \
0	53.0 ...	0.0
1	41.0 ...	0.0

2	2.0	...	0.0
3	0.0	...	0.0
4	82.0	...	0.0

	Disponibilité intérieure_kg	Exportations - Quantité_kg \
0	53000000.0	0.0
1	41000000.0	2000000.0
2	2000000.0	0.0
3	0.0	0.0
4	82000000.0	0.0

	Importations - Quantité_kg	Nourriture_kg	Pertes_kg	Production_kg \
0	0.0	53000000.0	0.0	53000000.0
1	40000000.0	39000000.0	2000000.0	3000000.0
2	2000000.0	2000000.0	0.0	0.0
3	0.0	0.0	0.0	0.0
4	82000000.0	82000000.0	0.0	0.0

	Semences_kg	Traitement_kg	Variation de stock_kg
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	0.0

[5 rows x 29 columns]

2.3 - Analyse exploratoire du fichier aide alimentaire

```
[70]: #Afficher les dimensions du dataset
print(f"Le tableau comporte {aide_alimentaire.shape[0]} observations ou
articles")
print(f"Le tableau comporte {aide_alimentaire.shape[1]} colonnes")
```

Le tableau comporte 1475 observations ou articles

Le tableau comporte 4 colonnes

```
[71]: #Affichage les 5 premières lignes de la table
display(aide_alimentaire.head())
```

	Pays bénéficiaire	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

```
[72]: #changement du nom de la colonne Pays bénéficiaire par Zone
aide_alimentaire = aide_alimentaire.rename(columns={'Pays_bénéficiaire': 'Zone'})
```

```
[73]: #Multiplication de la colonne Aide_alimentaire qui contient des tonnes par 1000
      ↪pour avoir des kg

aide_alimentaire['Valeur_kg'] = aide_alimentaire['Valeur'] * 1000
aide_alimentaire = aide_alimentaire.drop(columns=['Valeur'])
```

```
[74]: #Affichage les 5 premières lignes de la table
print(aide_alimentaire.head())
```

	Pays bénéficiaire	Année	Produit	Valeur_kg
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

2.4 - Analyse exploratoire du fichier sous nutrition

```
[75]: #Afficher les dimensions du dataset
print(f"Le tableau comporte {sous_nutrition.shape[0]} observations ou articles")
print(f"Le tableau comporte {sous_nutrition.shape[1]} colonnes")
```

Le tableau comporte 1218 observations ou articles
Le tableau comporte 3 colonnes

```
[76]: #Afficher les 5 premières lignes de la table
print(sous_nutrition.head())
```

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

```
[77]: #Conversion de la colonne (avec l'argument errors=coerce qui permet de
      ↪convertir automatiquement les lignes qui ne sont pas des nombres en NaN)
      #Puis remplacement des NaN en 0
sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur'],
      ↪errors='coerce')
sous_nutrition['Valeur'] = sous_nutrition['Valeur'].fillna(0)
```

```
[78]: #changement du nom de la colonne Valeur par sous_nutrition
sous_nutrition = sous_nutrition.rename(columns={'Valeur': 'sous_nutrition'})
```

```
[79]: #Multiplication de la colonne sous_nutrition par 1000000
sous_nutrition['sous_nutrition'] *=1000000
```

```
[80]: #Afficher les 5 premières lignes de la table
print(sous_nutrition.head())
```

	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8600000.0
1	Afghanistan	2013-2015	8800000.0
2	Afghanistan	2014-2016	8900000.0
3	Afghanistan	2015-2017	9700000.0
4	Afghanistan	2016-2018	10500000.0

3.1 - Proportion de personnes en sous nutrition en 2017

```
[81]: # FONCTIONS CREEES

POPULATION_MONDIALE_2017 = 7550000000
BESOIN_KCAL_MOYEN = 2500
besoin_mondial_kcal_jour = POPULATION_MONDIALE_2017 * BESOIN_KCAL_MOYEN
```

```
[82]: # Mapping de la colonne 'Année'

def calculer_annee_centrale(periode):
    # 1. Gestion des NaN et des types non-chaînes
    if pd.isna(periode):
        return periode

    # 2. Conversion en chaîne nécessaire pour le reste des opérations
    periode_str = str(periode).strip()

    try:
        # A. Cas 'YYYY-YYYY'
        if '-' in periode_str:
            annee_debut, annee_fin = map(int, periode_str.split('-'))
            return int((annee_debut + annee_fin) / 2)

        # B. Cas 'YYYY'
        return int(periode_str)

    except ValueError:
        return periode

sous_nutrition['Année'] = sous_nutrition['Année'].apply(calculer_annee_centrale)
sous_nutrition['Année'] = pd.to_numeric(sous_nutrition['Année'],
    errors='coerce').astype('Int64')

print(sous_nutrition['Année'].unique())
```



```
<IntegerArray>
[2013, 2014, 2015, 2016, 2017, 2018]
Length: 6, dtype: Int64
```

```
[83]: # Jointure entre la table population et la table sous nutrition, en ciblant
      ↪ l'année 2017

sous_nutrition = sous_nutrition.rename(columns={'sous_nutrition': 'Population'})
population_2017 = population[population['Année'] == 2017].copy()
sous_nutrition_2017 = sous_nutrition[sous_nutrition['Année'] == 2017].copy()

population_2017['Zone'] = population_2017['Zone'].str.strip()
sous_nutrition_2017['Zone'] = sous_nutrition_2017['Zone'].str.strip()

cle_jointure = ['Année', 'Zone']

df_2017 = pd.merge(
    population_2017,
    sous_nutrition_2017,
    on=cle_jointure,
    how='inner',
    suffixes=('_pop', '_sn'))

df_2017 = df_2017.rename(columns={
    'Population_pop': 'Population_Totale',
    'Population_sn': 'Sous_nutrition_Brute'})
```

```
[84]: #Affichage du dataset

print(f"Nombre de lignes dans df_2017 : {len(df_2017)}")
display(df_2017.head())
```

Nombre de lignes dans df_2017 : 203

	Zone	Année	Population_Totale	Sous_nutrition_Brute
0	Afghanistan	2017	36296113.0	10500000.0
1	Afrique du Sud	2017	57009756.0	3100000.0
2	Albanie	2017	2884169.0	100000.0
3	Algérie	2017	41389189.0	1300000.0
4	Allemagne	2017	82658409.0	0.0

```
[85]: #Calcul et affichage du nombre de personnes en état de sous nutrition

nombre_total_sous_nutrition = df_2017['Sous_nutrition_Brute'].sum()
pourcentage_sous_nutrition = (nombre_total_sous_nutrition /
    ↪ POPULATION_MONDIALE_2017) * 100
```

```
print(f"Le nombre TOTAL de personnes en état de sous-nutrition est :␣
↳{nombre_total_sous_nutrition:,.0f} personnes.")
print(f"Le pourcentage de personnes en sous-nutrition est de :␣
↳{pourcentage_sous_nutrition:.2f}%")
```

Le nombre TOTAL de personnes en état de sous-nutrition est : 535,700,000 personnes.

Le pourcentage de personnes en sous-nutrition est de : 7.10%

3.2 - Nombre théorique de personne qui pourrait être nourries en 2017

[86]: *#On commence par faire une jointure entre le data frame population et␣
↳Dispo_alimentaire afin d'ajouter dans ce dernier la population*

```
population_2017 = population[population['Année'] == 2017][['Zone',␣
↳'Population']].copy()
```

```
colonnes_population_a_joinre = ['Zone', 'Population']
cle_jointure = ['Zone']
```

```
dispo_alimentaire_avec_population = pd.merge(
    dispo_alimentaire,
    population_2017,
    on=cle_jointure,
    how='left')
```

[87]: *# Création de la colonne dispo_kcal avec calcul des kcal disponibles␣
↳mondialement*

```
dispo_alimentaire_avec_population['Dispo_Kcal_Totale_Jour'] = (
    dispo_alimentaire_avec_population['Disponibilité alimentaire (Kcal/personne/
↳jour)'] * (dispo_alimentaire_avec_population['Population']))
```

[88]: *# Analyse de la couverture calorique mondiale*

```
dispo_kcal_mondial_annuel_2017 =␣
↳dispo_alimentaire_avec_population['Dispo_Kcal_Totale_Jour'].sum()
pourcentage_couverture = (dispo_kcal_mondial_annuel_2017 /␣
↳besoin_mondial_kcal_jour) * 100

print(f"Disponibilité Totale Calculée : {dispo_kcal_mondial_annuel_2017:,.0f}␣
↳Kcal/jour")
print(f"Besoin Total de la Population : {besoin_mondial_kcal_jour:,.0f} Kcal/
↳jour")
print("---")
print(f"Couverture du Besoin Mondial : {pourcentage_couverture:,.0f} %")
```

Disponibilité Totale Calculée : 20,918,984,627,331 Kcal/jour

Besoin Total de la Population : 18,875,000,000,000 Kcal/jour

Couverture du Besoin Mondial : 111 %

```
[89]: #Calcul du nombre d'humains pouvant être nourris

personnes_nourries_theoriques = dispo_kcal_mondial_annuel_2017 /_
↳BESOIN_KCAL_MOYEN
excédent_deficit_kcal = dispo_kcal_mondial_annuel_2017 -_
↳besoin_mondial_kcal_jour
pourcentage_excédent_net = (dispo_kcal_mondial_annuel_2017 /_
↳besoin_mondial_kcal_jour) * 100

print(f"Potentiel à Nourrir : {personnes_nourries_theoriques / 1_000_000_000:,.
↳2f} Milliards de personnes (vs {POPULATION_MONDIALE_2017/1_000_000_000:,.2f}_
↳milliards réels)")
print(f"Pourcentage d'Excédent NET : {pourcentage_excédent_net:,.0f} %")
print("----")
if pourcentage_excédent_net > 0:
    print(" Conclusion : La totalité de la population mondiale peut être_
↳théoriquement nourrie en 2017.")
else :
    print(" Conclusion : La production mondiale est théoriquement insuffisante.
↳")
```

Potentiel à Nourrir : 8.37 Milliards de personnes (vs 7.55 milliards réels)

Pourcentage d'Excédent NET : 111 %

Conclusion : La totalité de la population mondiale peut être théoriquement nourrie en 2017.

3.3 - Nombre théorique de personne qui pourrait être nourrie avec les produits végétaux

```
[90]: #Transfert des données avec les végétaux dans un nouveau dataframe

print(dispo_alimentaire_avec_population['Origine'].unique())

filtre_vegetaux = dispo_alimentaire_avec_population['Origine'] == 'vegetale'
df_vegetaux = dispo_alimentaire_avec_population[filtre_vegetaux].copy()
```

['animale' 'vegetale']

```
[91]: # Calculer la Disponibilité Totale Calorique des Végétaux (Kcal/jour)

dispo_kcal_vegetal_2017 = df_vegetaux['Dispo_Kcal_Totale_Jour'].sum()
personnes_nourries_vegetal_theoriques = dispo_kcal_vegetal_2017 /_
↳BESOIN_KCAL_MOYEN
```

```

print(f"Disponibilité Totale Végétale : {dispo_kcal_vegetal_2017:,.0f} Kcal/
↳jour")
print(f"Quantité théorique pouvant être nourrie :␣
↳{personnes_nourries_vegetal_theoriques / 1_000_000_000:,.2f} Milliards de␣
↳personnes")

pourcentage_couverture_vegetal = (personnes_nourries_vegetal_theoriques /␣
↳POPULATION_MONDIALE_2017) * 100
print(f"Couverture de la Population Réelle : {pourcentage_couverture_vegetal:,.
↳1f} %")

```

Disponibilité Totale Végétale : 17,260,764,211,501 Kcal/jour
 Quantité théorique pouvant être nourrie : 6.90 Milliards de personnes
 Couverture de la Population Réelle : 91.4 %

3.4 - Utilisation de la disponibilité intérieure

```
[92]: COLONNE_DISPO = 'Disponibilité intérieure'
```

```

COLONNES_UTILISATION = [
    'Nourriture',
    'Aliments pour animaux',
    'Pertes',
    'Semences',
    'Traitement',
    'Autres Utilisations']

```

```
[93]: #Calcul de la disponibilité totale
```

```

dispo_kcal_totale = dispo_alimentaire_avec_population['Disponibilité␣
↳intérieure'].sum()

```

```
[94]: # Donner la part de l'alimentation humaine, animale et perdue (est-ce qu'on␣
↳peut aussi calculer les autres ?)
```

```

colonnes_a_sommer = [COLONNE_DISPO] + [col for col in COLONNES_UTILISATION if␣
↳col in dispo_alimentaire_avec_population.columns]

totaux_mondiaux = dispo_alimentaire_avec_population[colonnes_a_sommer].sum()
total_dispo_mondiale = totaux_mondiaux[COLONNE_DISPO]

print(" Répartition Mondiale de la Disponibilité Intérieure")
print("-----")

resultats_proportions = {}

for usage in COLONNES_UTILISATION:

```

```

    if usage in totaux_mondiaux.index:
        quantite_usage = totaux_mondiaux[usage]
        proportion = (quantite_usage / total_dispo_mondiale) * 100 if
↪total_dispo_mondiale != 0 else 0
        resultats_proportions[usage] = proportion

proportions_df = pd.Series(resultats_proportions).sort_values(ascending=False)

print("Catégorie d'Utilisation | Part Mondiale (%)")
print(":-|-----")

for usage, proportion in proportions_df.items():
    print(f"{usage:<23} | {proportion:>15,.2f}")

total_verifie = proportions_df.sum()
print("Total (Utilisations) | {:>15,.0f}%".format(total_verifie))

```

Répartition Mondiale de la Disponibilité Intérieure

```

-----
Catégorie d'Utilisation | Part Mondiale (%)
:-|-----
Nourriture | 49.51
Traitement | 22.38
Aliments pour animaux | 13.24
Autres Utilisations | 8.78
Pertes | 4.61
Semences | 1.57
Total (Utilisations) | 100%

```

3.5 - Utilisation des céréales

```

[95]: COLONNE_PRODUIT = 'Produit'
COLONNE_DISPO = 'Disponibilité intérieure'
COLONNES_UTILISATION = ['Nourriture', 'Aliments pour animaux', 'Traitement',
↪'Pertes', 'Semences', 'Autres Utilisations']
LISTE_CEREALES = ['Blé', 'Riz (Eq Blanchi)', 'Maïs', 'Orge', 'Sorgho',
↪'Millet', 'Avoine', 'Seigle', 'Céréales', 'Autres']

```

```

[96]: # Aggrégation

df_cereales = dispo_alimentaire_avec_population[
    dispo_alimentaire_avec_population[COLONNE_PRODUIT].isin(LISTE_CEREALES)].
↪copy()

df_cereales.loc[:, 'Somme_Utilisations'] = df_cereales[COLONNES_UTILISATION].
↪sum(axis=1)
df_cereales.loc[:, 'Ajustement_Statistique_MT'] = (
    df_cereales[COLONNE_DISPO] - df_cereales['Somme_Utilisations'])

```

```

COLONNES_SOMMES = [COLONNE_DISPO] + COLONNES_UTILISATION +
↳ ['Ajustement_Statistique_MT']

DIVISEUR = sommes_mondiales_par_cereale[COLONNE_DISPO]
COLONNES_PROPORTIONS_FINALES = []
COLONNES_PROPORTIONS_COURTES = []

MAPPING_NOMS_COURTS = {
    'Nourriture': 'P_Nourr',
    'Aliments pour animaux': 'P_Animaux',
    'Pertes': 'P_Pertes',
    'Semences': 'P_Semences',
    'Traitement': 'P_Traitement',
    'Autres Utilisations': 'P_Autres',
    'Ajustement_Statistique_MT': 'P_Ajust'}

sommes_mondiales_par_cereale = df_cereales.
↳ groupby(COLONNE_PRODUIT)[COLONNES_SOMMES].sum().reset_index()

for usage_col in COLONNES_UTILISATION + ['Ajustement_Statistique_MT']:
    usage_court = MAPPING_NOMS_COURTS.get(usage_col, usage_col.replace('_MT',
↳ ''))
    colonne_prop_nom_complet = f'Part_{usage_col.replace("_MT", "")} (%)'
    colonne_prop_nom_court = f'{usage_court} (%)'

    sommes_mondiales_par_cereale.loc[:, colonne_prop_nom_court] = (
        (sommes_mondiales_par_cereale[usage_col] / DIVISEUR) * 100).fillna(0)

    COLONNES_PROPORTIONS_COURTES.append(colonne_prop_nom_court)

sommes_mondiales_par_cereale.loc[:, 'Total (%)'] = (
    sommes_mondiales_par_cereale[COLONNES_PROPORTIONS_COURTES].sum(axis=1))

print(" Répartition Globale des Usages de Céréales (Somme Mondiale)")
print("----")

colonnes_affichage = [COLONNE_PRODUIT] + COLONNES_PROPORTIONS_COURTES + ['Total_
↳ (%)']

print(
    sommes_mondiales_par_cereale[colonnes_affichage]
    .sort_values(by='P_Nourr (%)', ascending=False)
    .to_markdown(index=False, floatfmt=".2f"))

```

```

Répartition Globale des Usages de Céréales (Somme Mondiale)
----

```

Produit	P_Nourr (%)	P_Animaux (%)	P_Traitement (%)	P_Pertes (%)	P_Semences (%)	P_Autres (%)	P_Ajust (%)	Total (%)
Riz (Eq Blanchi)	79.32	7.06	1.42	5.76	2.56	3.88	0.00	100.00
Millet	77.03	11.05	1.35	7.61	2.28	0.66	0.03	100.00
Blé	67.38	19.08	1.16	4.05	5.05	3.30	-0.02	100.00
Sorgho	41.47	42.60	6.34	4.72	1.35	3.52	-0.00	100.00
Seigle	33.21	48.89	6.76	3.39	7.62	0.12	0.01	100.00
Avoine	16.67	69.43	0.15	2.20	10.62	0.88	0.05	100.00
Maïs	13.10	57.14	4.99	4.26	0.72	19.81	-0.01	100.00
Orge	4.84	65.98	19.16	3.30	6.27	0.47	-0.01	100.00

```
[97]: import plotly.express as px
import pandas as pd
import numpy as np

COLONNES_GRAPHIQUE = [
    'P_Nourr (%)',
    'P_Animaux (%)',
    'P_Traitement (%)',
    'P_Pertes (%)']

df_graphique = results_df_lisible[[COLONNE_PRODUIT] + COLONNES_GRAPHIQUE].copy()

NOMS_LEGENDE = {
    'P_Nourr (%)': 'Nourriture (Humain)',
    'P_Animaux (%)': 'Aliments pour Animaux',
    'P_Traitement (%)': 'Traitement (Industrie)',
    'P_Pertes (%)': 'Pertes (Inefficacité)'}

df_graphique.rename(columns=NOMS_LEGENDE, inplace=True)
df_long = pd.melt(
    df_graphique,
    id_vars=[COLONNE_PRODUIT],
    value_vars=NOMS_LEGENDE.values(),
    var_name='Utilisation',
    value_name='Part (%)')
```

```

fig = px.bar(
    df_long,
    x=COLONNE_PRODUIT,
    y='Part (%)',
    color='Utilisation',
    barmode='group', # Barres côte à côte
    title='Comparaison des Usages Clés (Humain vs. Animal vs. Industrie) par
    ↪Céréale',
    color_discrete_map={
        'Nourriture (Humain)': '#4CAF50',
        'Aliments pour Animaux': '#FF9800',
        'Traitement (Industrie)': '#9C27B0',
        'Pertes (Inefficacité)': '#F44336'},

    category_orders={COLONNE_PRODUIT: results_df_lisible.
    ↪sort_values(by='P_Nourr (%)', ascending=False)[COLONNE_PRODUIT].tolist()})

fig.update_layout(
    xaxis_title="Céréale",
    yaxis_title="Part (%) de la Disponibilité Intérieure",
    legend_title_text="Usages Clés",
    hovermode="x unified")

fig.show()

```

3.5 - Utilisation animale

```

[98]: # Calculer la Disponibilité Totale Calorique animale (Kcal/jour)

df_animaux =
    ↪dispo_alimentaire_avec_population[dispo_alimentaire_avec_population['Origine']
    ↪== 'animale'].copy()
dispo_kcal_animal_2017 = df_animaux['Dispo_Kcal_Totale_Jour'].sum()
personnes_nourries_animal_theoriques = dispo_kcal_animal_2017 /
    ↪BESOIN_KCAL_MOYEN

print("  Potentiel Animal Mondial (2017) ")
print("----")
print(f"Disponibilité Totale Animale : {dispo_kcal_animal_2017:,.0f} Kcal/jour")
print(f"Potentiel Théorique : {personnes_nourries_animal_theoriques /
    ↪1_000_000_000:,.2f} Milliards de personnes")

pourcentage_couverture_animal = (personnes_nourries_animal_theoriques /
    ↪POPULATION_MONDIALE_2017) * 100
print(f"Couverture de la Population Réelle : {pourcentage_couverture_animal:,.
    ↪1f} %")

```


Potentiel Animal Mondial (2017)

Disponibilité Totale Animale : 3,658,220,415,830 Kcal/jour

Potentiel Théorique : 1.46 Milliards de personnes

Couverture de la Population Réelle : 19.4 %

[99]: *#Calcul de la proportion animale en kcal en pourcentage*

```
dispo_kcal_animal_2017 = df_animaux['Dispo_Kcal_Totale_Jour'].sum()
dispo_kcal_vegetal_2017 = df_vegetaux['Dispo_Kcal_Totale_Jour'].sum()

dispo_kcal_mondial_total = dispo_kcal_animal_2017 + dispo_kcal_vegetal_2017
proportion_animale = (dispo_kcal_animal_2017 / dispo_kcal_mondial_total) * 100
proportion_vegetale = (dispo_kcal_vegetal_2017 / dispo_kcal_mondial_total) * 100

print(f"Proportion d'Alimentation Animale : {proportion_animale:,.1f} %")
print(f"Proportion d'Alimentation Végétale : {proportion_vegetale:,.1f} %")
```

Proportion d'Alimentation Animale : 17.5 %

Proportion d'Alimentation Végétale : 82.5 %

3.6 - Pays avec la proportion de personnes sous-alimentée la plus forte en 2017

[100]: `df_2017.columns.tolist()`
`print(df_2017.head())`

	Zone	Année	Population_Totale	Sous_nutrition_Brute
0	Afghanistan	2017	36296113.0	10500000.0
1	Afrique du Sud	2017	57009756.0	3100000.0
2	Albanie	2017	2884169.0	100000.0
3	Algérie	2017	41389189.0	1300000.0
4	Allemagne	2017	82658409.0	0.0

[101]: *#affichage après trie des 10 pires pays*

```
df_2017['Proportion_Sous_Alimentee'] = (
    df_2017['Sous_nutrition_Brute'] / df_2017['Population_Totale']) * 100

df_trie = df_2017.sort_values(
    by='Proportion_Sous_Alimentee',
    ascending=False)

df_10_pires_pays = df_trie.head(10)
print(df_10_pires_pays[['Zone', 'Proportion_Sous_Alimentee',
    ↪ 'Sous_nutrition_Brute', 'Population_Totale']].to_string(index=False))
```

	Zone	Proportion_Sous_Alimentee
Sous_nutrition_Brute	Population_Totale	
	Haïti	48.259182

5300000.0	10982366.0		
République populaire démocratique de Corée			47.188685
12000000.0	25429825.0		
		Madagascar	41.062924
10500000.0	25570512.0		
		Libéria	38.279742
1800000.0	4702226.0		
		Lesotho	38.249438
800000.0	2091534.0		
		Tchad	37.957606
5700000.0	15016753.0		
		Rwanda	35.055619
4200000.0	11980961.0		
		Mozambique	32.810898
9400000.0	28649018.0		
		Timor-Leste	32.173531
400000.0	1243258.0		
		Afghanistan	28.928718
10500000.0	36296113.0		

3.7 - Pays qui ont le plus bénéficié d'aide alimentaire depuis 2013

```
[102]: print(aide_alimentaire.columns.tolist())
```

```
['Pays bénéficiaire', 'Année', 'Produit', 'Valeur_kg']
```

```
[103]: COLONNE_PAYS = 'Pays bénéficiaire'
COLONNE_AIDE = 'Valeur_kg'
COLONNE_ANNEE = 'Année'
```

```
[104]: #calcul du total de l'aide alimentaire par pays

ANNEE_DEBUT = 2013

aide_filtree = aide_alimentaire[aide_alimentaire[COLONNE_ANNEE] >= ANNEE_DEBUT] .
    ↪copy()

aide_par_pays = aide_filtree.groupby(COLONNE_PAYS)[COLONNE_AIDE].sum().
    ↪reset_index()
aide_par_pays = aide_par_pays.rename(columns={COLONNE_AIDE:
    ↪'Total_Aide_Alimentaire'})

total_aide_mondiale = aide_par_pays['Total_Aide_Alimentaire'].sum()
resultat_trie = aide_par_pays.sort_values(by='Total_Aide_Alimentaire',
    ↪ascending=False)

print(f"La quantité totale d'aide alimentaire distribuée depuis {ANNEE_DEBUT}
    ↪est de : {total_aide_mondiale:,.0f} kg")
```

```
print("----")
print(" Top 10 des Pays Bénéficiaires d'Aide Alimentaire (en kg)")

print(resultat_trie.head(10))
```

La quantité totale d'aide alimentaire distribuée depuis 2013 est de :
11,035,901,000 kg

Top 10 des Pays Bénéficiaires d'Aide Alimentaire (en kg)

	Pays bénéficiaire	Total_Aide_Alimentaire
50	République arabe syrienne	1858943000
75	Éthiopie	1381294000
70	Yémen	1206484000
61	Soudan du Sud	695248000
60	Soudan	669784000
30	Kenya	552836000
3	Bangladesh	348188000
59	Somalie	292678000
53	République démocratique du Congo	288502000
43	Niger	276344000

3.8 - Evolution des 5 pays qui ont le plus bénéficiés de l'aide alimentaire entre 2013 et 2016

```
[105]: #Création d'un dataframe avec la zone, l'année et l'aide alimentaire puis
        ↪groupby sur zone et année

df_aide_simple = aide_alimentaire[['Pays bénéficiaire', 'Année', 'Valeur_kg']].
        ↪copy()
aide_annuelle_par_pays = df_aide_simple.groupby(['Pays bénéficiaire',
        ↪'Année'])['Valeur_kg'].sum().reset_index()

aide_annuelle_par_pays = aide_annuelle_par_pays.rename(
    columns={'Valeur_kg': 'Aide_Totale_Annuelle'})

aide_filtree_2013_2016 = aide_annuelle_par_pays.loc[
    (aide_annuelle_par_pays['Année'] >= 2013) &
    (aide_annuelle_par_pays['Année'] <= 2016)].copy()

aide_totale_par_pays_periode = aide_filtree_2013_2016.groupby('Pays_
        ↪bénéficiaire')['Aide_Totale_Annuelle'].sum().reset_index()
aide_totale_par_pays_periode['Période'] = '2013-2016'
```

```
[106]: #Création de la liste de pays ayant le plus bénéficié d'aide alimentaire

aide_totale_par_pays_periode = aide_totale_par_pays_periode.sort_values(
    by='Aide_Totale_Annuelle',
    ascending=False)
```

```
print(" Aide Alimentaire TOTALE par Pays (avec Période)")
print("-----")
print(aide_totale_par_pays_periode.head(5))
```

```
Aide Alimentaire TOTALE par Pays (avec Période)
-----
      Pays bénéficiaire Aide_Totale_Annuelle Période
50 République arabe syrienne      1858943000 2013-2016
75 Éthiopie      1381294000 2013-2016
70 Yémen      1206484000 2013-2016
61 Soudan du Sud      695248000 2013-2016
60 Soudan      669784000 2013-2016
```

```
[107]: # Evolution de 2013 à 2016

top_5_pays_liste = aide_totale_par_pays_periode['Pays bénéficiaire'].head(5).
    ↪tolist()

df_top_5_evolution = aide_filtree_2013_2016[
    aide_filtree_2013_2016['Pays bénéficiaire'].isin(top_5_pays_liste)].copy()

evolution_pivot = df_top_5_evolution.pivot(
    index='Pays bénéficiaire',
    columns='Année',
    values='Aide_Totale_Annuelle')

evolution_pivot = evolution_pivot.fillna(0)
evolution_pivot.columns = evolution_pivot.columns.astype(str)

print(" Évolution de l'Aide Alimentaire par Pays (2013 - 2016)")
print("*(Unités : kg)*")
print("-----")

print(
    evolution_pivot
    .sort_values(by='2016', ascending=False)
    .to_markdown(floatfmt=".0f"))
```

```
Évolution de l'Aide Alimentaire par Pays (2013 - 2016)
*(Unités : kg)*
```

```
-----
| Pays bénéficiaire | 2013 | 2014 | 2015 | 2016 |
|:-----:|:-----:|:-----:|:-----:|:-----:|
| Yémen | 264764000 | 103840000 | 372306000 | 465574000 |
| République arabe syrienne | 563566000 | 651870000 | 524949000 | 118558000 |
| Soudan | 330230000 | 321904000 | 17650000 | 0 |
```

Soudan du Sud	196330000	450610000	48308000	0	
Éthiopie	591404000	586624000	203266000	0	

3.9 - Pays avec le moins de disponibilité par habitant

```
[108]: COLONNE_PAYS = 'Zone'
COLONNE_KCAL_PAR_PERSONNE_JOUR = 'Disponibilité alimentaire (Kcal/personne/
↳jour)'
```

```
[109]: #Calcul de la disponibilité en kcal par personne par jour par pays

dispo_alimentaire_avec_population[COLONNE_KCAL_PAR_PERSONNE_JOUR] = \
    pd.
    ↳to_numeric(dispo_alimentaire_avec_population[COLONNE_KCAL_PAR_PERSONNE_JOUR],
    ↳errors='coerce').fillna(0)

dispo_kcal_par_pays = dispo_alimentaire_avec_population.
    ↳groupby(COLONNE_PAYS)[COLONNE_KCAL_PAR_PERSONNE_JOUR].sum().reset_index()

dispo_kcal_par_pays = dispo_kcal_par_pays.rename(
    columns={COLONNE_KCAL_PAR_PERSONNE_JOUR:
    ↳'Dispo_Kcal_Totale_Par_Personne_Jour'})
```

```
[110]: #Affichage des 10 pays qui ont le moins de dispo alimentaire par personne

print("10 pays qui ont le moins de dispo alimentaire par personne")
print("-----")
resultat_trie = dispo_kcal_par_pays.sort_values(
    by='Dispo_Kcal_Totale_Par_Personne_Jour', ascending=True)

resultat_trie['Dispo_Kcal_Totale_Par_Personne_Jour'] = \
    resultat_trie['Dispo_Kcal_Totale_Par_Personne_Jour'].map('{:,.0f}'.format)

display(resultat_trie.head(10))
```

10 pays qui ont le moins de dispo alimentaire par personne

```
-----
Zone \
128 République centrafricaine
166 Zambie
91 Madagascar
0 Afghanistan
65 Haïti
133 République populaire démocratique de Corée
151 Tchad
167 Zimbabwe
114 Ouganda
154 Timor-Leste
```

	Dispo_Kcal_Totale_Par_Personne_Jour
128	1,879
166	1,924
91	2,056
0	2,087
65	2,089
133	2,093
151	2,109
167	2,113
114	2,126
154	2,129

3.10 - Pays avec le plus de disponibilité par habitant

```
[111]: #Affichage des 10 pays qui ont le plus de dispo alimentaire par personne

print("10 pays qui ont le plus de dispo alimentaire par personne")
print("-----")
resultat_trie = dispo_kcal_par_pays.sort_values(
    by='Dispo_Kcal_Totale_Par_Personne_Jour', ascending=False)

resultat_trie['Dispo_Kcal_Totale_Par_Personne_Jour'] = \
    resultat_trie['Dispo_Kcal_Totale_Par_Personne_Jour'].map('{:,.0f}'.format)

display(resultat_trie.head(10))
```

10 pays qui ont le plus de dispo alimentaire par personne

	Zone	Dispo_Kcal_Totale_Par_Personne_Jour
11	Autriche	3,770
16	Belgique	3,737
159	Turquie	3,708
171	États-Unis d'Amérique	3,682
74	Israël	3,610
72	Irlande	3,602
75	Italie	3,578
89	Luxembourg	3,540
168	Égypte	3,518
4	Allemagne	3,503

```
[112]: # Répartition impot/export pays sur-nutrition

PAYS_CIBLES = ['Autriche', 'Belgique', 'États-Unis d\'Amérique']

df_top_pays = dispo_alimentaire_avec_population[
    dispo_alimentaire_avec_population[COLONNE_PAYS].isin(PAYS_CIBLES)].copy()
```

```

colonnes_a_sommer = COLONNES_FLUX + COLONNES_USAGE + [COLONNE_DISPO]
bilan_par_pays = df_top_pays.groupby(COLONNE_PAYS)[colonnes_a_sommer].sum().
↳reset_index()

df_repartition_usages = bilan_par_pays[[COLONNE_PAYS]].copy()

for col in COLONNES_USAGE:
    nom_col_prop = f'Part {col} (%)'
    df_repartition_usages[nom_col_prop] = (
        bilan_par_pays[col] / bilan_par_pays[COLONNE_DISPO] * 100).round(2)

df_repartition_flux = bilan_par_pays[[COLONNE_PAYS]].copy()

df_repartition_flux['Part Importations/DI (%)'] = (
    bilan_par_pays['Importations - Quantité'] / bilan_par_pays[COLONNE_DISPO] *
↳100).round(2)

df_repartition_flux['Poids Exportations/DI (%)'] = (
    bilan_par_pays['Exportations - Quantité'] / bilan_par_pays[COLONNE_DISPO] *
↳100).round(2)

display(df_repartition_usages)
display(df_repartition_flux)

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[112], line 8
      3 PAYS_CIBLES = ['Autriche', 'Belgique', 'États-Unis d\'Amérique']
      5 df_top_pays = dispo_alimentaire_avec_population[
      6     dispo_alimentaire_avec_population[COLONNE_PAYS].isin(PAYS_CIBLES)].
↳copy()
----> 8 colonnes_a_sommer = COLONNES_FLUX + COLONNES_USAGE + [COLONNE_DISPO]
      9 bilan_par_pays = df_top_pays.groupby(COLONNE_PAYS)[colonnes_a_sommer].
↳sum().reset_index()
     11 df_repartition_usages = bilan_par_pays[[COLONNE_PAYS]].copy()

NameError: name 'COLONNES_FLUX' is not defined

```

3.11 - Exemple de la Thaïlande pour le Manioc

```

[ ]: PAYS_CIBLE = 'Thaïlande'
COLONNE_POURCENTAGE = 'Proportion_Sous_Alimentee'
COLONNE_PAYS = 'Zone'
PRODUIT_CIBLE = 'Manioc'
COLONNE_KCAL_TOTALE = 'Dispo_Kcal_Totale_Par_Personne_Jour'

```

```
[ ]: #création d'un dataframe avec uniquement la Thaïlande

sous_nutrition_thaïlande = df_2017[df_2017[COLONNE_PAYS] == PAYS_CIBLE].copy()
```

```
[ ]: #Calcul de la sous nutrition en Thaïlande

sous_nutrition_thaïlande = df_2017[df_2017[COLONNE_PAYS] == PAYS_CIBLE].copy()
pourcentage_thaïlande = sous_nutrition_thaïlande[COLONNE_POURCENTAGE].iloc[0]

print(f"La proportion de la population sous-alimentée est de :␣
↳{pourcentage_thaïlande:.2f} %")
```

```
[ ]: # On calcule la proportion exportée en fonction de la proportion globale

donnees_thaïlande_manioc = dispo_alimentaire_avec_population[
    (dispo_alimentaire_avec_population['Zone'] == PAYS_CIBLE) &
    (dispo_alimentaire_avec_population['Produit'] == PRODUIT_CIBLE)].copy()

print(f" Analyse du Manioc en {PAYS_CIBLE}")
print("-----")

colonnes_a_analyser = [
    'Production',
    'Exportations - Quantité',
    'Importations - Quantité',
    'Nourriture',
    'Pertes',
    'Aliments pour animaux',
    'Disponibilité intérieure',
    'Autres Utilisations']

if not donnees_thaïlande_manioc.empty:
    colonnes_existantes = [col for col in colonnes_a_analyser if col in␣
↳donnees_thaïlande_manioc.columns]

    print(donnees_thaïlande_manioc[colonnes_existantes].
↳to_markdown(index=False))

else:
    print(f"Aucune donnée trouvée pour le {PRODUIT_CIBLE} en {PAYS_CIBLE}.␣
↳Vérifiez l'orthographe ou le filtre.")
```

```
[ ]: # Disponibilité alimentaire en Thaïlande

disponibilite_thaïlande = dispo_kcal_par_pays[dispo_kcal_par_pays['Zone'] ==␣
↳PAYS_CIBLE].copy()
```



```

if not disponibilite_thailande.empty:
    dispo_kcal_valeur = disponibilite_thailande[COLONNE_KCAL_TOTALE].iloc[0]

    print(f"La disponibilité totale par habitant (en Kcal/jour) est de :␣
    ↳{dispo_kcal_valeur} Kcal/jour")
else:
    print(f"Aucune donnée trouvée pour la {PAYS_CIBLE} dans le DataFrame␣
    ↳'dispo_kcal_par_pays'.")

```

Étape 6 - Analyse complémentaires